Min Heap in C++

```
#include <iostream>
using namespace std;

#define MAX 100

class MinHeap {
  int heap[MAX];
  int size;

public:
  MinHeap() {
    size = 0;
  }

  // Insert element
  void insert(int val) {
    if (size == MAX) {
      cout << "Heap Overflow\n";
      return;
    }

    int i = size;
    heap[size++] = val;

    // Heapify Up
```

```
    while (i != 0 && heap[(i - 1) / 2] > heap[i]) {
        swap(heap[i], heap[(i - 1) / 2]);
        i = (i - 1) / 2;
    }
}

// Delete minimum element (root)
void deleteMin() {
    if (size <= 0) {
        cout << "Heap Underflow\n";
        return;
    }

    if (size == 1) {
        size--;
        return;
    }

    // Replace root with last element
    heap[0] = heap[size - 1];
    size--;

    heapify(0);
}

// Heapify (Downward)
void heapify(int i) {
    int smallest = i;
```

```
        int left = 2 * i + 1;
        int right = 2 * i + 2;

        if (left < size && heap[left] < heap[smallest])
            smallest = left;

        if (right < size && heap[right] < heap[smallest])
            smallest = right;

        if (smallest != i) {
            swap(heap[i], heap[smallest]);
            heapify(smallest);
        }
    }

    // Display heap
    void display() {
        for (int i = 0; i < size; i++)
            cout << heap[i] << " ";
        cout << endl;
    }
};

// MAIN
int main() {
    MinHeap h;

    h.insert(10);
```

```
    h.insert(20);
    h.insert(5);
    h.insert(15);

    cout << "Heap elements: ";
    h.display();

    h.deleteMin();

    cout << "After deleting min: ";
    h.display();

    return 0;
}
```

Output

```
Heap elements: 5 15 10 20
After deleting min: 10 15 20


=== Code Execution Successful ===
```